

CS 4391/5391 – Reinforcement Learning

Homework 4

Department of Computer Science
Whitacre College of Engineering
Texas Tech University

Due by Friday, May 8, 11:59pm via Canvas

Submission Policy: It is **strongly** recommended to work in groups. As a team, you can work in parallel on different components. You must be signed up for a group on Canvas and include a brief section in your report describing the specific contributions of each team member (it is perfectly acceptable to state that the team worked jointly on all tasks).

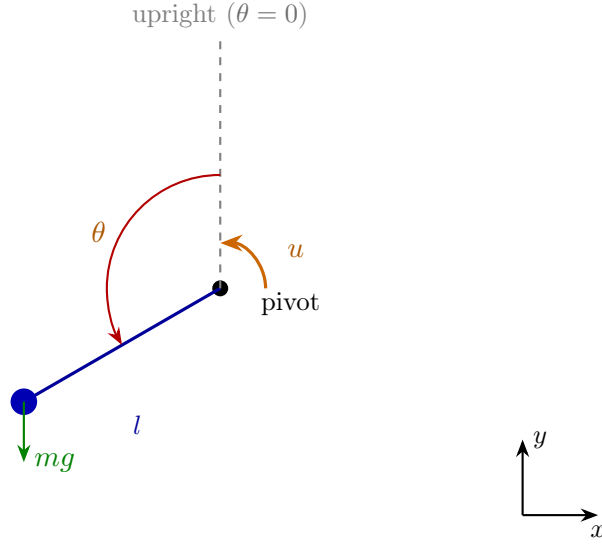
The use of generative AI to solve this assignment—especially for writing code—is strictly prohibited. Any violation will result in failing the course. Please feel free to reach out to the course staff with any questions. Gaining hands-on experience is an essential part of the learning process.

Introduction

In this assignment, you will implement the Proximal Policy Optimization (PPO) algorithm and apply it to solve the task of swinging up an inverted pendulum—a continuous dynamical system with continuous state s and action a spaces.

The Pendulum Swing-Up Problem

The inverted pendulum is a paradigmatic benchmark in control theory and reinforcement learning. The agent must swing a pendulum from a hanging position to an upright, balanced configuration using limited torque—requiring coordinated energy injection across multiple swings. This problem is a foundational step toward upright stabilization of bipedal robots, self-righting maneuvers in legged locomotion, and balancing in humanoid systems.



A uniform rod of mass m and length l is attached to a fixed pivot. A control torque u is applied at the pivot. The angle θ is measured from the **upright** position ($\theta = 0$ = balanced on top).

1) Equation of Motion

For a uniform rod (moment of inertia $I = \frac{1}{3}ml^2$, center of mass at $l/2$):

$$\ddot{\theta}(t) = \frac{3g}{2l} \sin(\theta(t)) + \frac{3}{ml^2} u(t) \quad (1)$$

The positive gravitational term reflects that θ is measured from the *unstable* equilibrium: gravity drives the pendulum *away* from $\theta = 0$.

2) State-Space Representation

With $x_1 = \theta$, $x_2 = \dot{\theta}$:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ \frac{3g}{2l} \sin(x_1) \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{3}{ml^2} \end{bmatrix} u \quad (2)$$

3) Discrete-Time Dynamics

Gymnasium Pendulum-v1 uses a **semi-implicit Euler** scheme—velocity is updated first, then the new velocity is used to update position:

$$\boxed{\begin{aligned} x_2[k+1] &= x_2[k] + \Delta t \left(\frac{3g}{2l} \sin(x_1[k]) + \frac{3}{ml^2} u[k] \right) \\ x_1[k+1] &= x_1[k] + \Delta t \cdot x_2[k+1] \end{aligned}} \quad (3)$$

Note: the position update uses the *already-updated* $x_2[k+1]$. Angular velocity is clipped to $[-8, 8]$ rad/s.

4) Gymnasium Pendulum-v1 Parameters

Parameter	Symbol	Default
Mass	m	1.0 kg
Length	l	1.0 m
Gravity	g	10.0 m/s ²
Time step	Δt	0.05 s
Max torque	$u_{\max}$	2.0 N·m
Max angular velocity	$\dot{\theta}_{\max}$	8.0 rad/s
Episode length	—	200 steps

Observation: $s = [\cos \theta, \sin \theta, \dot{\theta}]$. Reward: $r = -(\theta^2 + 0.1 \dot{\theta}^2 + 0.001 u^2)$, with $\theta \in [-\pi, \pi]$.

Policy and Action Distribution

Recalling the state $s = [x_1, x_2] = [\theta, \dot{\theta}]$, actions are torques at the pivot, sampled from a Gaussian:

$$a \sim \mathcal{N}(\mu(s), \sigma), \quad \text{cf.,} \quad a \sim \mathcal{N}(\mu(s), \sigma(s)) \text{ in the general case.}$$

In the basic implementation you may use a state-independent σ as a manually tuned hyperparameter.

You are provided with utilities (see `Modules.py`) to help implement PPO following the pseudocode in the PPO paper (Schulman et al., 2017). Feel free to refer to the code from Homework 1 as inspiration. Do not copy end-to-end implementations from the web—that will not contribute to understanding the algorithm (and is easily detected).

An additional resource: OpenAI Spinning Up: Key Concepts and Terminology.

1 Task 1: Environment Interaction Loop

Your first objective is to set up the environment interaction loop and experience replay buffer for the Pendulum environment.

To-Do:

1. Implement an environment interaction loop for the Pendulum environment.
2. Create a policy network (feed-forward neural network) that receives the state s and returns the mean $\mu(s)$ and $\log \sigma$ (natural logarithm of the standard deviation) of actions. You may use a state-independent standard deviation for the basic implementation.
3. Create and test an **experience replay buffer** using a random policy (i.e., a Gaussian distribution with arbitrary, randomly initialized weights of the policy network).
4. Implement an **episode reward processing function** that converts one-step rewards into discounted rewards-to-go:

$$R(s_t) = \sum_{k=0}^{T-t} \gamma^k r_{t+k},$$

which is the discounted cumulative reward starting from state s_t .

2 Task 2: Vanilla Policy Gradient

In this task, you will implement a basic policy gradient agent before adding the PPO enhancements.

To-Do:

1. Implement a **vanilla policy gradient** agent. The gradient ascent steps are computed using the average of the gradient of the log-likelihood over a trajectory, weighted by the rewards-to-go from each state.
2. Try **different step sizes** (learning rates) in the gradient ascent.
3. **Visualize:** Plot learning curves (accumulated discounted reward per episode vs. training episode) for at least two different learning rates.
4. **Analysis:** Explain your observations regarding the effect of step size on learning stability and convergence.

3 Task 3: Critic Network and Generalized Advantage Estimation

Next, you will add a value function baseline and generalized advantage estimation (GAE) to reduce variance in the policy gradient.

To-Do:

1. Add a feed-forward network for the **critic**, accepting the state $s = [\sin(\theta), \cos(\theta), \dot{\theta}]$ and returning a scalar estimate of the value $V(s)$.
2. Implement **Generalized Advantage Estimation** (GAE), as described in Equations 11–12 of the PPO paper, to be used instead of raw rewards-to-go.
3. **Visualize:** Plot learning curves comparing vanilla policy gradient (rewards-to-go) against the GAE-based variant.
4. **Analysis:** Explain how GAE affects the learning process and compare with the results from Task 2.

4 Task 4: PPO Surrogate Objective

Now you will implement the core PPO clipped surrogate objective.

To-Do:

1. Implement the **surrogate objective** for the policy gradient as described in Equation 7 of the PPO paper, both **with and without clipping**.
2. Implement the **total loss** as described in Equation 9 of the PPO paper.
3. **Visualize:** Plot learning curves and loss curves comparing the clipped vs. unclipped surrogate objectives.
4. **Analysis:** Explain the role of clipping and how it affects training stability.

5 Task 5: Full PPO Algorithm

Finally, you will combine all components into the full PPO algorithm (Algorithm 1 in the PPO paper).

To-Do:

1. Combine all components from Tasks 1–4 to implement **Algorithm 1** from the PPO paper. In your basic implementation, you may collect data with a single actor ($N = 1$).
2. Run the full PPO algorithm and tune hyperparameters as needed.
3. **Visualize:** Produce the following plots:
 - **Learning curves:** Accumulated discounted reward by policy at the k -th training episode, for each algorithm setting (e.g., with/without clipping, with/without GAE).
 - **Loss curves:** Total loss (Equation 9) for each algorithm setting.
4. **Analysis:** Discuss the final performance, the effect of each component (GAE, clipping), and any hyperparameter sensitivity you observed.

Optional: Extending HW4 to the Final Project

To extend this assignment into a final project, complete **any four** of the following extensions:

- **Ensemble critics:** Use an ensemble of B critics instead of a single critic: $V_{\text{ens}}(s) = \frac{1}{B} \sum_{b=1}^B V_b(s)$, where each $V_b(s)$ is parametrized by a separate network.
- **Parallel data collection:** Collect data in parallel across multiple pendulum environments. Report the improvement in wall-clock training time.
- **Partial observability with recurrence:** Add recurrence to the policy and train in an environment with partial observability (e.g., use only the angular velocity rather than the full state).
- **Image-based observations:** Use images instead of state vectors. Since velocity is not directly available, you will need to either stack consecutive frames or use a recurrent policy.
- **State-dependent standard deviation:** Train $\log \sigma(s)$ as a function of the state, rather than using a fixed hyperparameter.
- **Your own idea** (please confirm with Dr. Tiomkin before implementing).

Deliverables

Please submit your work using one of the following formats: either a compiled PDF report paired with your runnable Python (.py) source files, or a single, self-contained Python Notebook (.ipynb) that includes both your code and written analysis. Please **zip** all your files into a single archive and submit on Canvas.

Important Note for Notebook Submissions: If submitting an .ipynb file, all cell outputs must be pre-rendered and visible in the submitted file. To avoid missing outputs, please “Restart & Run All” and save the notebook right before uploading. Submissions will be evaluated based on the visible outputs present in the file.

1. **Runnable Python code** (.py format) that cleanly reproduces all your results, plots, and environments.
2. The following items (included in your PDF or Notebook markdown):
 - **Team Contribution Statement.**
 - **Task 1:** Environment loop and replay buffer implementation, with a brief description of your design choices.
 - **Task 2:** Learning curves for vanilla policy gradient with different learning rates, plus your written analysis.
 - **Task 3:** Learning curves comparing rewards-to-go vs. GAE, plus your written analysis.
 - **Task 4:** Learning curves and loss curves comparing clipped vs. unclipped surrogate objectives, plus your written analysis.
 - **Task 5:** Final learning curves and loss curves for the full PPO algorithm across different settings, plus your comprehensive analysis.